Jobsheet 9 – Abstract Class dan Interface

I. Kompetensi

Setelah menyelesaikan lembar kerja ini mahasiswa diharapkan mampu:

1. Menjelaskan maksud dan tujuan penggunaan *abstract class* dan *interface*;
2. Menerapkan *abstract class* di dalam pembuatan program;
3. Menerapkan *interface* dan *multiple interface* di dalam pembuatan program;
4. Membedakan kapan menggunakan *abstract class* dan kapan menggunakan *interface*.

Alokasi Waktu

Jobsheet ini dirancang untuk diselesaikan dalam **200 menit**. Gunakan pembagian waktu berikut sebagai panduan.

Kegiatan	Waktu
Membaca pendahuluan dan menyiapkan project	10 menit
Percobaan 1: Abstract Class (termasuk pertanyaan)	40 menit
Percobaan 2: Interface (termasuk pertanyaan)	45 menit
Percobaan 3: Multiple Interface (termasuk pertanyaan)	40 menit
Tugas	65 menit
Total	**200 menit**

II. Pendahuluan

A. Abstract Class

Abstract class adalah class yang **tidak dapat diinstansiasi** (tidak bisa dibuat objeknya dengan `new`), namun dapat di-*extend*. *Abstract class* baru dapat dimanfaatkan ketika ia di-*extend* oleh class lain.

Karakteristik:

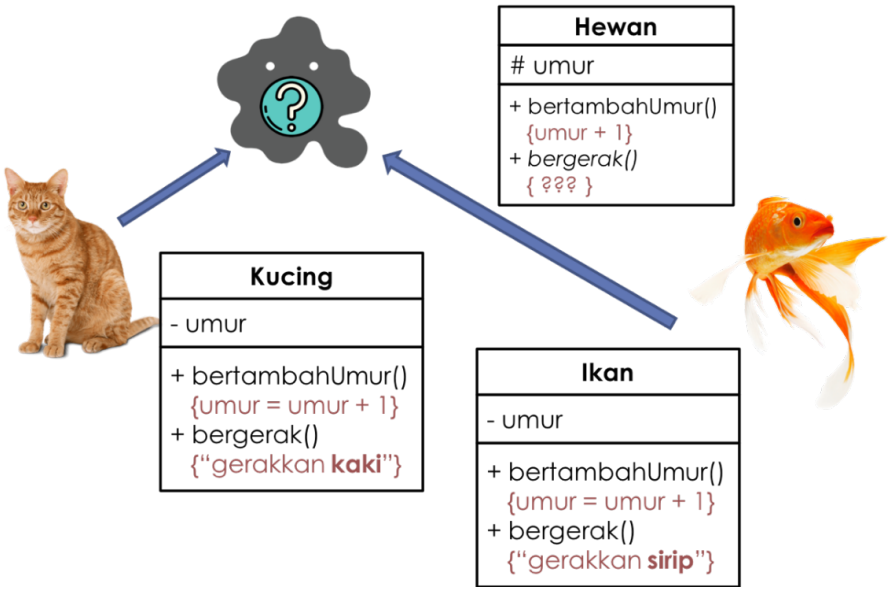
a. Dapat memiliki atribut, *constructor*, dan *method* biasa seperti class pada umumnya.

b. Dapat memiliki *method* yang tidak memiliki badan (hanya deklarasinya saja), disebut *abstract method*. Class yang memiliki minimal satu *abstract method* **wajib** dideklarasikan sebagai *abstract class*.

c. Dideklarasikan dengan kata kunci `abstract class`.

d. Class turunan (yang bukan abstract) **wajib** meng-*override* semua *abstract method* milik induknya.

Kegunaan: menggambarkan sesuatu yang bersifat umum, yang hanya bisa berfungsi setelah dideskripsikan ke dalam bentuk yang lebih spesifik. Contoh: semua hewan pasti bisa bergerak, tetapi cara bergeraknya baru jelas setelah kita tahu hewannya apa.



Gambar 1. Ilustrasi abstract class Hewan

B. Interface

Interface adalah **kontrak** berisi daftar *method* yang **harus** dipenuhi oleh suatu class agar class tersebut dapat dianggap sebagai "sesuatu yang lain". Contoh: seorang Sarjana dapat disebut *Cumlaude* jika ia memenuhi kontrak `ICumlaude`, yaitu bisa `lulus()` dan `meraihIPKTinggi()`.

Karakteristik:

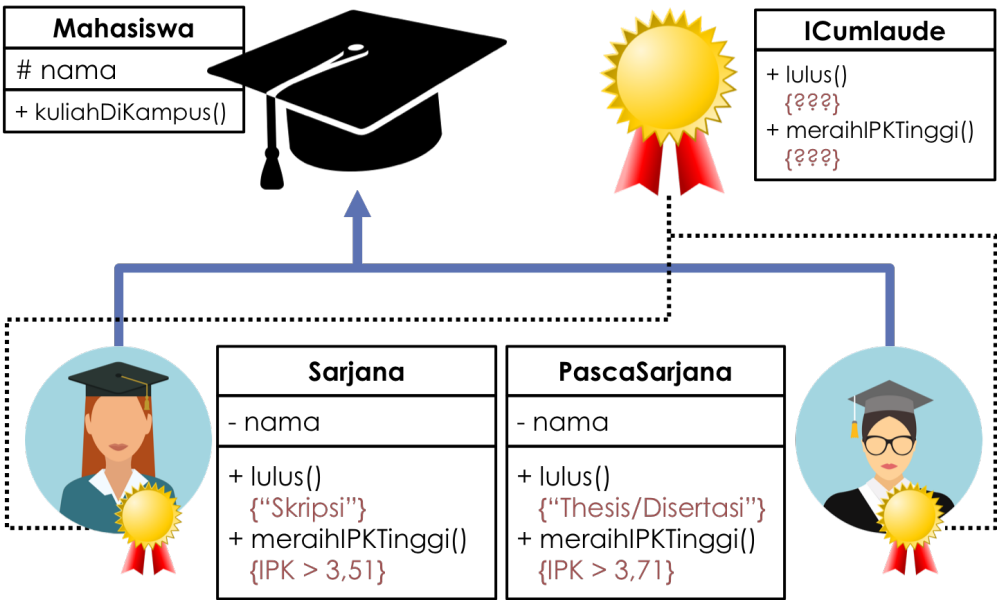
a. Dideklarasikan dengan kata kunci `interface` dan digunakan oleh class dengan kata kunci `implements`.

b. *Method* di dalam interface otomatis bersifat `public abstract` (boleh tidak ditulis). Sejak Java 8, interface juga boleh memiliki `default` dan `static` method yang memiliki badan, tetapi pada jobsheet ini kita fokus pada *abstract method*.

c. Interface tidak boleh memiliki variabel biasa; yang boleh hanya **konstanta** (otomatis `public static final`).

d. Interface tidak memiliki *constructor* dan **tidak dapat dibuat objeknya** dengan `new`.

e. Pada jobsheet ini nama interface diawali huruf kapital 'I' (contoh: `ICumlaude`) agar mudah dibedakan dari class. Ini kebiasaan penamaan, bukan aturan wajib Java; library Java sendiri memakai nama seperti `Comparable` dan `Runnable`.



Gambar 2. Ilustrasi interface ICumlaude

Multiple interface. Sebuah class hanya boleh meng-*extend* **satu** class, tetapi boleh meng-*implements* **lebih dari satu** interface (dipisah koma), misalnya `class PascaSarjana extends Mahasiswa implements ICumlaude, IBerprestasi`. Inilah cara Java memberi sebuah class beberapa "kemampuan" sekaligus, sebagai pengganti *multiple inheritance*.

Aturan yang sering menyebabkan error:

a. *Method* hasil implementasi interface pada class **harus** `public`.

b. Nama, jumlah, dan tipe parameter *method* pada class harus sama persis dengan yang ada di interface.

c. *Method* dan konstanta di dalam interface tidak boleh diberi modifier `private` atau `protected`.

d. Interface dan *abstract class* tidak dapat dibuat objeknya dengan `new`; objek dibuat dari class yang meng-*implements*/meng-*extend*-nya.

C. Notasi UML

- *Abstract class*: nama class dan *abstract method* ditulis **miring** (boleh ditambah keterangan «abstract»).
- *Interface*: diawali notasi «interface» di atas nama; nama interface **tidak** miring.
- *Extends* (pewarisan): garis penuh dengan kepala panah segitiga kosong, mengarah ke class induk.
- *Implements*: garis **putus-putus** dengan kepala panah segitiga kosong, mengarah ke interface.

D. Perbedaan Abstract Class dan Interface

Abstract Class	Interface
Dapat berisi *abstract method* dan *method* biasa (memiliki badan)	Berisi *abstract method* (sejak Java 8 juga boleh `default` dan `static` method)
Modifier *method* ditulis sendiri (`public`, `protected`, dll.)	*Method* otomatis `public abstract`, tidak perlu ditulis

Abstract Class	Interface
Dapat memiliki atribut biasa (*instance variable*) dan konstanta	Hanya dapat memiliki konstanta (otomatis `public static final`)
Dapat memiliki *constructor*	Tidak memiliki *constructor*
Sebuah class hanya dapat meng-*extend* **satu** class (termasuk abstract class)	Sebuah class dapat meng-*implements* **lebih dari satu** interface
Abstract class dapat meng-*implements* satu atau lebih interface	Interface dapat meng-*extend* interface lain, tetapi tidak dapat meng-*implements* apa pun
Hubungan "**adalah**" (Kucing adalah Hewan)	Hubungan "**dapat/bisa**" (Sarjana dapat menjadi Cumlaude)

III. Praktikum

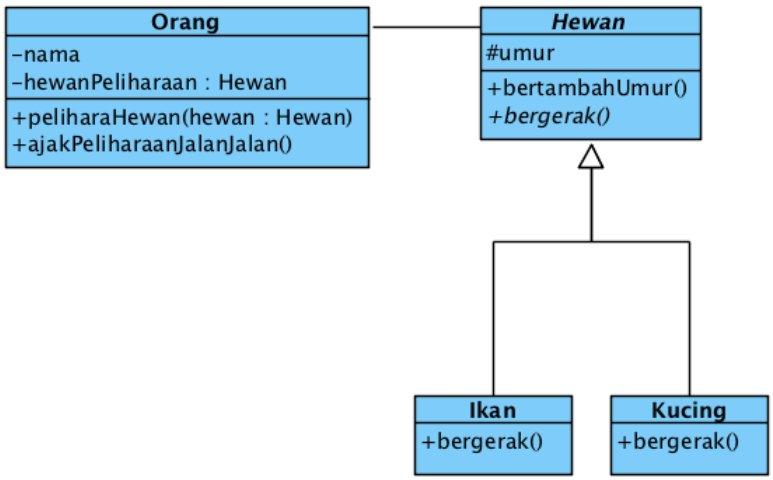
Persiapan Project

Buatlah sebuah project baru di NetBeans dengan nama **PBOMinggu9**. Di dalam project tersebut buat **dua package**: `abstractclass` untuk Percobaan 1 dan `interfacelatihan` untuk Percobaan 2 dan 3 (klik kanan pada *Source Packages* → **New** → **Java Package…**). Setiap class/interface dibuat dengan klik kanan pada package → **New** → **Java Class…** atau **New** → **Java Interface…**.

Tips: setelah menuliskan `extends` atau `implements`, NetBeans menampilkan ikon lampu peringatan. Klik ikon tersebut lalu pilih **Implement all abstract methods** agar kerangka *method* dibuat otomatis, kemudian ganti isi `throw new UnsupportedOperationException(...)` dengan kode pada jobsheet.

Percobaan 1: Abstract Class (± 40 menit)

Semua hewan memiliki karakteristik yang sama, misalnya umur: hewan apa pun umurnya bertambah satu setiap tahun. Namun, setiap hewan memiliki cara **bergerak** yang berbeda: kucing melangkahkan kakinya, sedangkan ikan menggerakkan siripnya. Setiap orang yang memelihara hewan dapat mengajak peliharaannya berjalan-jalan, tetapi cara peliharaannya bergerak berbeda-beda. Skenario ini kita buat dengan memanfaatkan **abstract class**.



Gambar 3. Class diagram Percobaan 1

1. Pada package `abstractclass`, buat class **Hewan** sebagai *abstract class* berikut.

```
package abstractclass;

public abstract class Hewan {
    protected int umur;

    protected Hewan() {
        this.umur = 0;
    }

    public void bertambahUmur() {
        this.umur += 1;
    }

    public abstract void bergerak();
}
```

   Class `Hewan` berisi atribut dan *method* biasa, ditambah *abstract method* `bergerak()` yang tidak memiliki badan. *Method* ini nantinya wajib di-*override* oleh setiap class turunan `Hewan`.

2. Buat class **Kucing** yang meng-*extend* `Hewan`. Gunakan ikon lampu → **Implement all abstract methods**, lalu ubah isi *method* `bergerak()` menjadi seperti berikut.

```
package abstractclass;

public class Kucing extends Hewan {
    @Override
    public void bergerak() {
        System.out.println("Berjalan dengan KAKI, \"tap..tap..\"");
    }
}
```

3. Dengan cara yang sama, buat class **Ikan**.

```
package abstractclass;

public class Ikan extends Hewan {
    @Override
    public void bergerak() {
        System.out.println("Berenang dengan SIRIP, \"wush..wush..\"");
    }
}
```

4. Buat class biasa **Orang**. Class ini adalah pengguna dari *abstract class* `Hewan`.

```
package abstractclass;

public class Orang {
    private String nama;
    private Hewan hewanPeliharaan;

    public Orang(String nama) {
        this.nama = nama;
    }

    public void peliharaHewan(Hewan hewan) {
        this.hewanPeliharaan = hewan;
    }

    public void ajakPeliharaanJalanJalan() {
        System.out.println("Namaku " + this.nama);
        System.out.println("Hewan peliharaanku berjalan dengan cara: ");
        this.hewanPeliharaan.bergerak();
        System.out.println("----------------------------------------");
    }
}
```

5. Buat *main class* **Program** di package yang sama.

```
package abstractclass;

public class Program {
    public static void main(String[] args) {
        Kucing kucingKampung = new Kucing();
        Ikan lumbaLumba = new Ikan();

        Orang ani = new Orang("Ani");
        Orang budi = new Orang("Budi");

        ani.peliharaHewan(kucingKampung);
        budi.peliharaHewan(lumbaLumba);

        ani.ajakPeliharaanJalanJalan();
        budi.ajakPeliharaanJalanJalan();
    }
}
```

6. Jalankan class `Program` (klik kanan → **Run File** atau Shift + F6), lalu amati hasilnya.

Output:

```
Namaku Ani
Hewan peliharaanku berjalan dengan cara:
Berjalan dengan KAKI, "tap..tap.."
----------------------------------------
Namaku Budi
Hewan peliharaanku berjalan dengan cara:
Berenang dengan SIRIP, "wush..wush.."
----------------------------------------
```

Pertanyaan Percobaan 1

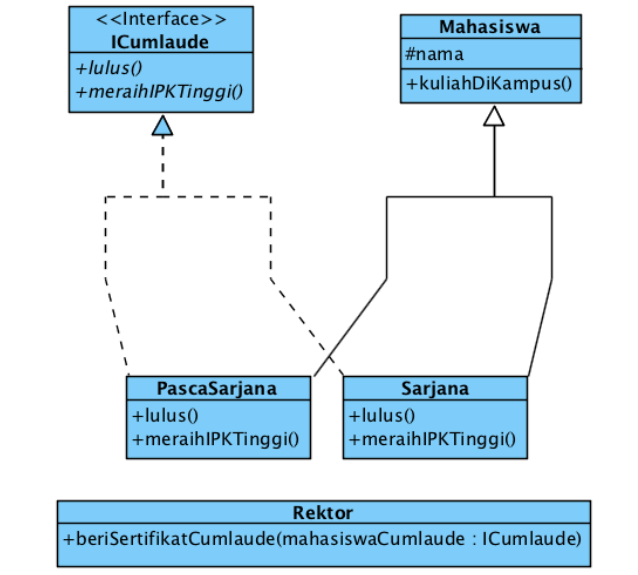
Setelah setiap uji coba, kembalikan kode seperti semula sebelum mengerjakan nomor berikutnya.

1. Jelaskan jalannya program di atas. Mengapa pemanggilan `this.hewanPeliharaan.bergerak()` di class `Orang` menghasilkan output yang berbeda untuk Ani dan Budi?
2. Tambahkan baris `Hewan hewan = new Hewan();` pada method `main()`. Tunjukkan hasil kompilasinya dan jelaskan secara singkat.

3. Bolehkah sebuah class yang meng-*extend abstract class* tidak mengimplementasikan *abstract method* induknya? Buktikan dengan menghapus *method* `bergerak()` dari class `Ikan`, lalu tunjukkan hasil kompilasinya dan jelaskan.
4. Tambahkan *abstract method* `public abstract void bernapas();` di dalam class `Ikan` tanpa mengubah deklarasi class `Ikan`. Tunjukkan hasil kompilasinya dan jelaskan aturan yang dilanggar.

Percobaan 2: Interface (± 45 menit)

Pada sebuah wisuda, Rektor akan memberikan sertifikat *Cumlaude* kepada semua mahasiswa yang memenuhi syarat. Mahasiswa **Sarjana** harus menyelesaikan **skripsi** dan memiliki IPK lebih dari 3,51, sedangkan mahasiswa **Pasca Sarjana** harus menyelesaikan **tesis** dan memiliki IPK lebih dari 3,71. Persyaratan ini kita tuangkan dalam interface `ICumlaude`.



Gambar 4. Class diagram Percobaan 2

1. Pada package `interfacelatihan`, buat interface **ICumlaude** (klik kanan → **New** → **Java Interface…**) dengan dua *abstract method*: `lulus()` dan `meraihIPKTinggi()`.

```
package interfacelatihan;

public interface ICumlaude {
    public abstract void lulus();
    public abstract void meraihIPKTinggi();
}
```

2. Buat class **Mahasiswa**.

```
package interfacelatihan;

public class Mahasiswa {
    protected String nama;

    public Mahasiswa(String nama) {
        this.nama = nama;
    }

    public void kuliahDiKampus() {
        System.out.println("Aku mahasiswa, namaku " + this.nama);
        System.out.println("Aku berkuliah di kampus.");
    }
}
```

3. Buat class **Sarjana** yang meng-*extend* `Mahasiswa` dan meng-*implements* `ICumlaude`. Gunakan ikon lampu untuk membuat kerangka *method*, lalu sesuaikan isinya.

```
package interfacelatihan;

public class Sarjana extends Mahasiswa implements ICumlaude {
    public Sarjana(String nama) {
        super(nama);
    }

    @Override
    public void lulus() {
        System.out.println("Aku sudah menyelesaikan SKRIPSI");
    }

    @Override
    public void meraihIPKTinggi() {
        System.out.println("IPK-ku lebih dari 3,51");
    }
}
```

`Sarjana` meng-*extend* `Mahasiswa`, artinya **Sarjana adalah Mahasiswa**. Agar objek `Sarjana` dapat disebut **Cumlaude**, ia harus meng-*implements* interface `ICumlaude`.

4. Dengan cara yang sama, buat class **PascaSarjana**.

```
package interfacelatihan;

public class PascaSarjana extends Mahasiswa implements ICumlaude {
    public PascaSarjana(String nama) {
        super(nama);
    }

    @Override
    public void lulus() {
        System.out.println("Aku sudah menyelesaikan TESIS");
    }

    @Override
    public void meraihIPKTinggi() {
        System.out.println("IPK-ku lebih dari 3,71");
    }
}
```

5. Buat class **Rektor**. Perhatikan bahwa parameter *method* `beriSertifikatCumlaude()` bertipe interface `ICumlaude`.

```
package interfacelatihan;

public class Rektor {
    public void beriSertifikatCumlaude(ICumlaude mahasiswaCumlaude) {
        System.out.println("Saya REKTOR, memberikan sertifikat cumlaude.");
        System.out.println("Selamat! Silakan perkenalkan diri Anda..");

        mahasiswaCumlaude.lulus();
        mahasiswaCumlaude.meraihIPKTinggi();

        System.out.println("----------------------------------------");
    }
}
```

6. Buat *main class* **Program** di package `interfacelatihan`.

```
package interfacelatihan;

public class Program {
    public static void main(String[] args) {
        Rektor pakRektor = new Rektor();

        Mahasiswa mhsBiasa = new Mahasiswa("Charlie");
        Sarjana sarjanaCumlaude = new Sarjana("Dini");
        PascaSarjana masterCumlaude = new PascaSarjana("Elok");

        pakRektor.beriSertifikatCumlaude(mhsBiasa);
        pakRektor.beriSertifikatCumlaude(sarjanaCumlaude);
        pakRektor.beriSertifikatCumlaude(masterCumlaude);
    }
}
```

7. Jika semua class diketik dengan benar, akan muncul **error** dan program tidak dapat dijalankan. Perbaiki kode Anda sehingga program menghasilkan output seperti berikut.

Output:

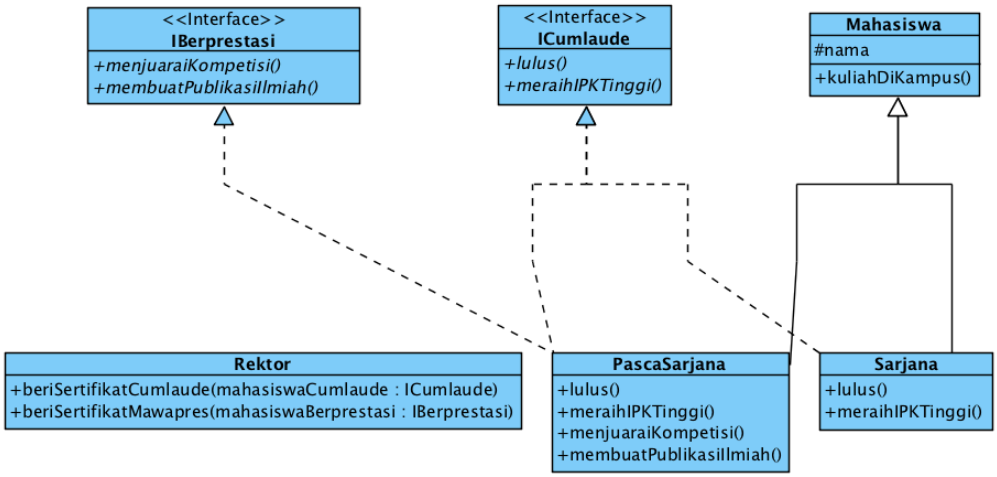
```
Saya REKTOR, memberikan sertifikat cumlaude.
Selamat! Silakan perkenalkan diri Anda..
Aku sudah menyelesaikan SKRIPSI
IPK-ku lebih dari 3,51
----------------------------------------
Saya REKTOR, memberikan sertifikat cumlaude.
Selamat! Silakan perkenalkan diri Anda..
Aku sudah menyelesaikan TESIS
IPK-ku lebih dari 3,71
----------------------------------------
```

Pertanyaan Percobaan 2

1. Mengapa pada langkah 7 terjadi error? Tuliskan pesan error-nya dan jelaskan bagaimana Anda memperbaikinya.
2. Dapatkah *method* `kuliahDiKampus()` dipanggil dari objek `sarjanaCumlaude` di class `Program`? Mengapa demikian?
3. Dapatkah *method* `kuliahDiKampus()` dipanggil dari parameter `mahasiswaCumlaude` di *method* `beriSertifikatCumlaude()` pada class `Rektor`? Mengapa demikian?
4. Hapus kata kunci `public` pada *method* `lulus()` di class `Sarjana`. Tunjukkan hasil kompilasinya dan jelaskan, lalu kembalikan seperti semula.

Percobaan 3: Multiple Interface (± 40 menit)

Pada acara wisuda yang sama, Rektor juga akan memberikan **sertifikat Mawapres** (Mahasiswa Berprestasi). Kriteria berprestasi adalah **menjuarai kompetisi** dan **membuat publikasi ilmiah**. Kriteria ini dituangkan dalam interface baru `IBerprestasi`, sehingga `PascaSarjana` nantinya meng-*implements* dua interface sekaligus. Percobaan ini melanjutkan kode Percobaan 2 pada package yang sama.



Gambar 5. Class diagram Percobaan 3

1. Pada package `interfacelatihan`, buat interface baru **IBerprestasi**.

```
package interfacelatihan;

public interface IBerprestasi {
    public abstract void menjuaraiKompetisi();
    public abstract void membuatPublikasiIlmiah();
}
```

2. Ubah deklarasi class **PascaSarjana** dengan menambahkan `IBerprestasi` setelah kata kunci `implements` (dipisah koma). Gunakan ikon lampu untuk membuat kerangka *method*-nya, lalu tambahkan isi kedua *method* tersebut seperti berikut.

```
public class PascaSarjana extends Mahasiswa
        implements ICumlaude, IBerprestasi {
    // constructor, lulus(), dan meraihIPKTinggi() tetap seperti sebelumnya

    @Override
    public void menjuaraiKompetisi() {
        System.out.println("Saya telah menjuarai kompetisi INTERNASIONAL");
    }

    @Override
    public void membuatPublikasiIlmiah() {
        System.out.println("Saya menerbitkan artikel di jurnal INTERNASIONAL");
    }
}
```

3. Tambahkan *method* `beriSertifikatMawapres()` pada class **Rektor**.

```
public void beriSertifikatMawapres(IBerprestasi mahasiswaBerprestasi) {
    System.out.println("Saya REKTOR, memberikan sertifikat MAWAPRES.");
    System.out.println("Selamat! Bagaimana Anda bisa berprestasi?");

    mahasiswaBerprestasi.menjuaraiKompetisi();
    mahasiswaBerprestasi.membuatPublikasiIlmiah();

    System.out.println("----------------------------------------");
}
```

4. Ubah *method* `main()` pada class **Program**: jadikan komentar semua baris yang memanggil `beriSertifikatCumlaude()`, lalu tambahkan pemanggilan `beriSertifikatMawapres()` seperti berikut.

```
public static void main(String[] args) {
    Rektor pakRektor = new Rektor();

    Sarjana sarjanaCumlaude = new Sarjana("Dini");
    PascaSarjana masterCumlaude = new PascaSarjana("Elok");

    // pakRektor.beriSertifikatCumlaude(sarjanaCumlaude);
    // pakRektor.beriSertifikatCumlaude(masterCumlaude);

    pakRektor.beriSertifikatMawapres(sarjanaCumlaude);
    pakRektor.beriSertifikatMawapres(masterCumlaude);
}
```

5. Akan terdapat **error** pada langkah 4 sehingga program tidak dapat dijalankan. Perbaiki kode program sehingga hasil eksekusinya sama seperti berikut.

Output:

```
Saya REKTOR, memberikan sertifikat MAWAPRES.
Selamat! Bagaimana Anda bisa berprestasi?
Saya telah menjuarai kompetisi INTERNASIONAL
Saya menerbitkan artikel di jurnal INTERNASIONAL
----------------------------------------
```

Pertanyaan Percobaan 3

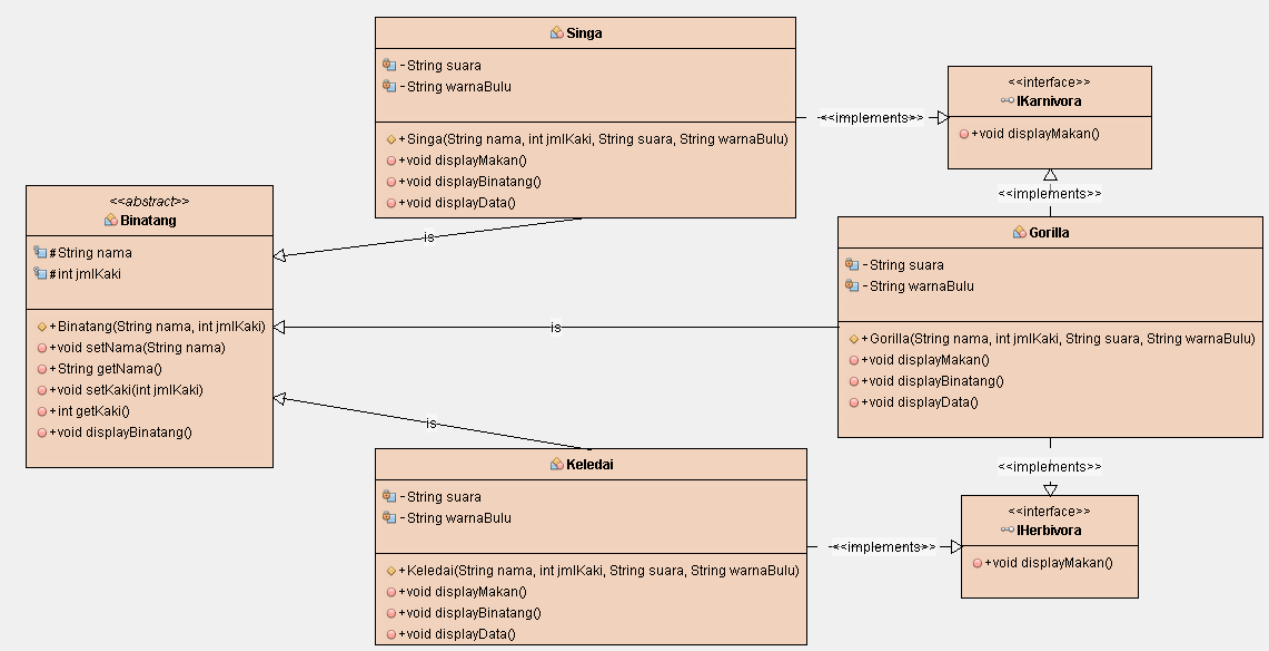
1. Pada interface `IBerprestasi`, ganti modifier `public` pada *method* `menjuaraiKompetisi()` menjadi `protected` dan pada *method* `membuatPublikasiIlmiah()` menjadi `private`. Apa yang terjadi? Jelaskan alasannya (sertakan *capture* hasilnya), lalu kembalikan seperti semula.
2. Tambahkan baris `IBerprestasi prestasi = new IBerprestasi();` pada *method* `main()`. Mengapa baris tersebut error?
3. Apabila **Sarjana Berprestasi** harus **menjuarai kompetisi NASIONAL** dan **menerbitkan artikel di jurnal NASIONAL**, modifikasilah class-class yang terkait agar di class `Program` objek `pakRektor` dapat memberikan sertifikat mawapres kepada objek `sarjanaCumlaude`, dengan output seperti berikut.

Output yang diharapkan:

```
Saya REKTOR, memberikan sertifikat MAWAPRES.
Selamat! Bagaimana Anda bisa berprestasi?
Saya telah menjuarai kompetisi NASIONAL
Saya menerbitkan artikel di jurnal NASIONAL
---------------------------------------
Saya REKTOR, memberikan sertifikat MAWAPRES.
Selamat! Bagaimana Anda bisa berprestasi?
Saya telah menjuarai kompetisi INTERNASIONAL
Saya menerbitkan artikel di jurnal INTERNASIONAL
---------------------------------------
```

IV. Tugas (± 65 menit)

Terdapat UML diagram dari sistem yang mengidentifikasi karakteristik dan jenis makanan beberapa binatang seperti gambar di bawah ini. **Singa** hanya makan daging sehingga tergolong **Karnivora**, sedangkan **Keledai** hanya mengonsumsi tumbuhan sehingga tergolong **Herbivora**. **Gorilla** dapat mengonsumsi kedua jenis makanan tersebut (**omnivora**), sehingga dikategorikan sebagai **kombinasi Herbivora dan Karnivora**. Setiap binatang juga memiliki karakteristik **nama, suara, warna bulu, dan jumlah kaki**.



Gambar 6. Class diagram Tugas

Berdasarkan penjelasan kasus dan UML di atas, implementasikan sistem tersebut dalam Java (buat package baru `tugas`) dengan output seperti berikut.

Output yang diharapkan:

```
Jenis: Herbivora
Makanan: Tumbuhan
Nama: Keledai
Jumlah Kaki: 4
Suara: Hehehehe
Warna Bulu: Abu-Abu

Jenis: Karnivora + Herbivora
Makanan: Daging + Tumbuhan
Nama: Gulali
Jumlah Kaki: 4
Suara: Haaum Hauum
Warna Bulu: Hitam Manis

Jenis: Karnivora
Makanan: Daging
Nama: CingaCing
Jumlah Kaki: 4
Suara: Roaar Roaaar
Warna Bulu: Coklat
```

Petunjuk: `Binatang` adalah *abstract class* (hubungan "adalah"), sedangkan `IKarnivora` dan `IHerbivora` adalah interface (hubungan "dapat"). Class `Gorilla` meng-*implements* kedua interface sekaligus. Karena kedua interface memiliki *method* `displayMakan()` dengan *signature* yang sama, `Gorilla` cukup menuliskan **satu** *method* `displayMakan()`.

Yang dikumpulkan: laporan berisi *capture* kode program beserta penjelasannya, *capture* output program, jawaban seluruh pertanyaan percobaan, dan UML dari sistem Anda.

========== Selamat Mengerjakan ==========